



STORM CHASER

Mobile Application — Assessment Submission

Rebecca Maguire

March 12, 2026

Mobile Assessment

React + TypeScript

Weather + Camera +
Local Storage

This document presents a polished overview of the Storm Chaser mobile assessment project: what the app does, how it meets the brief, the architectural decisions behind it, and the key submission items still to finalize before handoff.

Project Snapshot

App Goal	A mobile-first weather and storm documentation app for hobbyist meteorologists and storm chasers.
Primary Stack	React, TypeScript, Vite, Tailwind CSS, custom hooks, Open-Meteo API, Capacitor Camera integration, IndexedDB + localStorage persistence.
Core Views	Weather, Document, Log, and Map.
Submission Intent	A concise, readable assessment project emphasizing functionality, clean structure, reusable components, and practical mobile UX.

Key Features: Weather API · Camera · Geolocation · Local Storage · Map View · Forecasts · Skeleton Screens

Assessment Coverage

The app was developed to align closely with the uploaded Speer Technologies Mobile Development Assessment brief, including the required weather, documentation, persistence, navigation, testing, and documentation expectations.

Requirement	Status	Implementation
Current weather by device location	Complete	Uses device geolocation and Open-Meteo current forecast endpoint.
Storm-relevant weather metrics	Complete	Displays temperature, wind speed/direction, humidity, precipitation, pressure, cloud cover, and apparent temperature.
"Not Found" weather state	Complete	Dedicated WeatherNotFound component with retry action when retrieval fails.
Storm photo capture + metadata	Complete	Camera/gallery capture with weather conditions, coordinates, date/time, notes, and storm classification.
Local persistence	Complete	IndexedDB primary storage with localStorage fallback/backup for saved reports.
Navigation / intuitive UI	Complete	Bottom mobile navigation across Weather, Document, Log, and Map sections.
Unit test	Complete	Vitest suite passing with 2 files and 4 tests.
Bonus enhancements	Included	Forecast views, saved photo gallery, map view, skeleton loading, pull-to-refresh-style manual refresh, dark theme presentation.

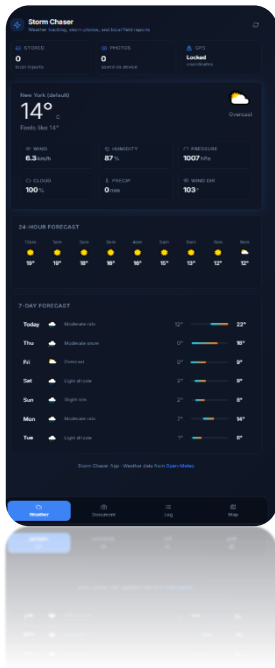
What stands out

- Mobile-focused presentation rather than a generic desktop weather dashboard.
- Weather retrieval and failure handling are explicitly separated through dedicated loading and error-state components.
- Storm reports are documented with photos and metadata rather than plain text logging.
- Persistence is implemented with a more credible storage approach than a quick in-memory demo.
- The app includes several extras beyond the minimum brief, including forecast surfaces, a photo gallery, and a location map.

Practical read: for a non-senior submission, the project is already comfortably at the point where polish, documentation, and presentation matter more than piling on additional features for the sake of movement.

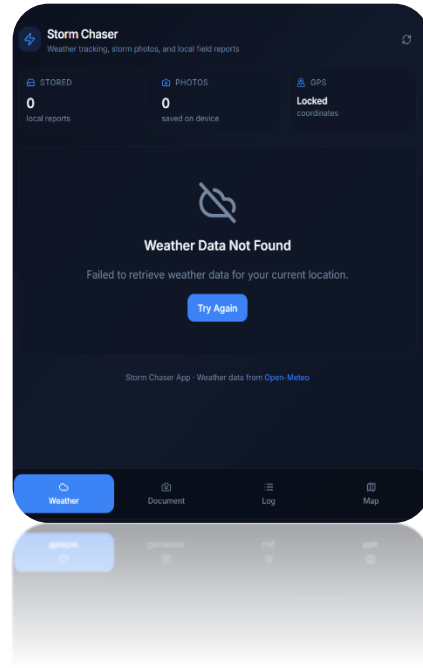
SCREENSHOT WALKTHROUGH

Weather Experience



Weather Dashboard

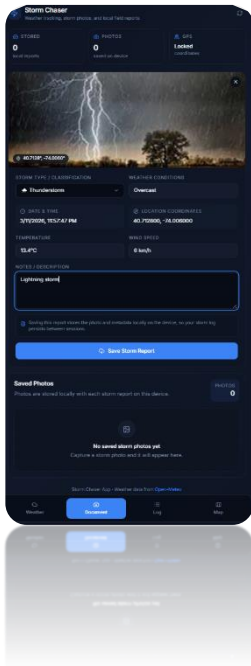
Live weather view with device-location forecast and forecast cards.



Weather Error State

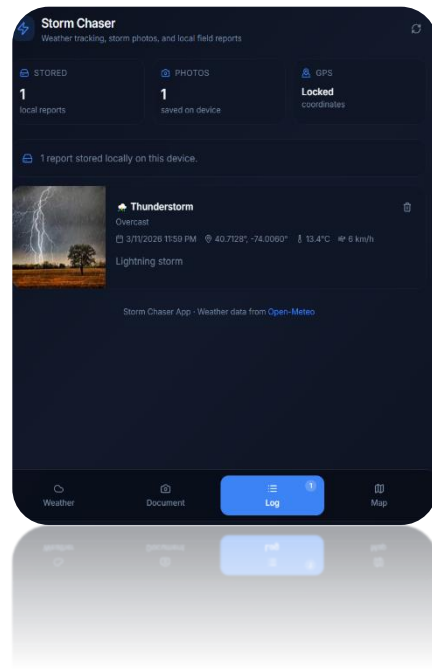
WeatherNotFound component with retry action.

Photo Upload and Storm Report Flow



Storm Documentation

Document tab showing photo upload, live weather metadata, storm classification, and save action.



Saved Photos / Log Evidence

Saved report evidence with persisted photo count / gallery / log entry / deletion option.

Architecture & Design Decisions

The implementation follows a straightforward component-and-hooks architecture: page orchestration at the top level, logic separated into reusable hooks, and view-specific concerns isolated into modular UI components.

Layer	Responsibility
Page orchestration	Index.tsx controls tab state, top-level refresh actions, and the composition of Weather, Document, Log, and Map sections.
Hooks	use-geolocation handles device coordinates, use-weather handles API retrieval/loading/error state, and use-storm-reports handles saved-report state and persistence.
Persistence	storm-report-storage.ts persists reports locally using IndexedDB as primary storage with localStorage as a fallback/backup layer.
Storm capture flow	StormDocForm coordinates camera/gallery import, image preparation/compression, metadata capture, and report submission.
Display surfaces	CurrentWeather, DailyForecast, HourlyForecast, PhotoGallery, StormLog, and StormMap each focus on one display responsibility.

Implementation notes

- The weather experience was designed around fast readability: current conditions first, supporting forecasts after, with dedicated empty/loading/error states.
- The documentation flow prioritizes one clear action: capture, classify, annotate, save.
- The UI leans dark and atmospheric to fit the storm-chasing theme without becoming visually noisy.
- A small test suite was added with Vitest to satisfy the brief's explicit request for at least one unit test.

Testing, QA, and Submission Packaging

Item	Status / Note
Unit tests	Passing: 2 test files, 4 tests.
README	Describes features, implementation decisions, testing, and known limitations.
AI Use	ChatGPT
Screen recording	A short 40 second UI walkthrough showing the core user journey: weather retrieval, storm documentation, photo handling, local save behavior, and evidence of persistence through the saved gallery/log views.
Submission PDF	Includes candidate name, assessment name, GitHub link, screen recording link, and time log.
Repository	Private and shared with developer@speer.io.

- The app meets the required mobile weather + storm documentation brief.
- The code is organized around reusable hooks and components rather than one oversized page file.
- Local persistence is implemented, not mocked.
- Error, loading, and saved-state experiences were handled intentionally.
- Testing and documentation were included to satisfy the brief beyond pure UI work.

Final Handoff Page

Submission Field	Value
Candidate Full Name	Rebecca Maguire
Assessment Name	Mobile - Android / Mobile Development Assessment
GitHub Repository Link	https://github.com/Kyla-Zeit/storm-chaser-app
Screen Recording Link	https://drive.google.com/file/d/1GulhZ7-GLZxD1NAg0Mih6RqcKA83h1Ks/view?usp=sharing
Time Spent	9 hr 45 min

AI Usage Disclosure Template

Tools used: ChatGPT

Used for: debugging assistance, code iteration, architecture suggestions, UI refinement, documentation drafting

AI-assisted portions: AI was used selectively for troubleshooting and polishing. Core app functionality, integration, testing, and final implementation decisions were completed manually.

Time Log

Task	Time Spent
Project setup and structure	1 hr 00 min
Weather integration	1 hr 00 min
Storm documentation flow	2 hr 15 min
Persistence and saved reports	2 hr 00 min
UI polish and navigation	1 hr 30 min
Testing and documentation	2 hr 00 min
Total	9 hr 45 min